Python Unit 2 – (4 marks)

1. Define a function in Python. Write and explain the general syntax of a user-defined function.

1. A function is a block of reusable code that performs a specific task. It runs only when it is called.
2. Functions help in reducing code repetition. They improve readability and modularity of the program.
3. In Python, a user-defined function is created using the def keyword. It is followed by the function name and parentheses.
4. General syntax:
   def function_name(parameters):
   statements
   return value
5. The parameters are inputs given to the function. The return statement sends the result back to where the function was called.

---

2. List any four built-in Python functions and explain their use.

1. print() is used to display output on the screen. It helps in showing messages or results.
2. len() is used to find the length of a string, list, or other object. It returns the total number of elements.
3. sum() is used to add all elements of an iterable like a list. It returns the total sum.
4. type() is used to check the data type of a variable. It tells whether the value is int, string, list, etc.
5. max() returns the largest value from a group of numbers. It is useful for comparisons.

---

3. Differentiate between built-in functions and user-defined functions with examples.

1. Built-in functions are already available in Python. User-defined functions are created by the programmer.
2. Built-in functions do not require definition. User-defined functions must be defined using def.
3. Example of built-in function: len("Python"). It returns 6.
4. Example of user-defined function:
   def greet():
   print("Hello")
5. Built-in functions save time and effort. User-defined functions are used when we need custom tasks.

---

4. Explain the purpose of the import statement in Python. Give examples.

1. The import statement is used to include a module in a program. It allows us to use functions and features from that module.
2. Modules help organize code into separate files. They make programs easier to manage and reuse.
3. Example: import math. This allows us to use mathematical functions like math.sqrt().
4. Example: import random. This allows us to generate random numbers using random.randint().
5. Without import, we cannot use functions from external modules. Import loads the module into the program.

5. Write a Python function to find the square of a number.

1. A function to find the square of a number multiplies the number by itself. It returns the result to the user.
2. We use the def keyword to create the function. The function name can be anything meaningful like square.
3. Example program:

```
def square(num):
   return num * num
print(square(4))
```

4. In this function, num is the parameter that receives the input value. The return statement sends back the square of the number.
5. When we call square(4), the function calculates 4 × 4. The output will be 16.

---

6. Explain any four functions of the math module with examples.

1. math.sqrt(x) is used to find the square root of a number. For example, math.sqrt(25) gives 5.
2. math.pow(x, y) is used to raise a number to a power. For example, math.pow(2, 3) gives 8.
3. math.ceil(x) returns the smallest integer greater than or equal to a number. For example, math.ceil(4.2) gives 5.
4. math.floor(x) returns the largest integer less than or equal to a number. For example, math.floor(4.8) gives 4.
5. To use these functions, we must first write import math. After importing, we can access them using dot notation like math.sqrt().

---

7. Write a program using the random module to generate a random number between 1 and 100.

1. The random module is used to generate random numbers in Python. It is useful in games, simulations, and testing.
2. To use it, we must first import the module. This is done by writing import random.
3. Example program:

```
import random
num = random.randint(1, 100)
print(num)
```

4. The function randint(1, 100) generates a random integer between 1 and 100. Both 1 and 100 are included in the range.
5. Every time the program runs, it may produce a different number. This is because the number is generated randomly.

---

8. Describe the use of the datetime module in Python.

1. The datetime module is used to work with date and time in Python. It helps in managing real-world date and time operations.
2. It provides classes like date, time, datetime, and timedelta. These classes help in creating and manipulating date and time objects.
3. We must import the module before using it. This is done by writing from datetime import datetime or import datetime.
4. The function datetime.now() is used to get the current date and time. The function date.today() is used to get the current date.
5. The datetime module is useful for scheduling, logging, and time calculations. It also helps in formatting date and time into readable strings.

9. Write examples of any two string module functions.

1. The string module provides constants and helper functions related to strings. We must import it first using import string.
2. One example is string.ascii_lowercase which gives all lowercase letters from a to z. For example, print(string.ascii_lowercase) prints abcdefghijklmnopqrstuvwxyz.
3. Another example is string.digits which gives numeric characters from 0 to 9. For example, print(string.digits) prints 0123456789.
4. These constants are useful for password generation and validation. They help avoid writing letters or digits manually.
5. The string module mainly provides predefined string constants. It does not modify strings like built-in string methods do.

---

10. List any four built-in functions in Python and explain their utility.

1. len() is used to find the length of an object like a string or list. For example, len("Python") returns 6.
2. type() is used to check the data type of a variable. For example, type(10) returns <class 'int'>.
3. range() is used to generate a sequence of numbers. For example, range(1,5) generates numbers from 1 to 4.
4. input() is used to take input from the user. It reads the data as a string.
5. These built-in functions are already available in Python. They make programming easier and reduce extra coding effort.

---

11. Differentiate between import math and from math import sqrt.

1. import math imports the whole math module. We must use dot notation to access functions like math.sqrt().
2. from math import sqrt imports only the sqrt function from the math module. We can directly use sqrt() without writing math.
3. When we use import math, all functions inside math are available. But we must write the module name before calling them.
4. When we use from math import sqrt, only the sqrt function is available. Other functions like pow() will not work unless imported.
5. Both statements are used to access math functions. The difference is in how we call and import the functions.

---

12. How do random.random(), random.randint(), and random.choice() differ?

1. random.random() returns a random float value between 0 and 1. The value is always a decimal number.
2. random.randint(a, b) returns a random integer between a and b. Both a and b are included in the result.
3. random.choice(seq) selects a random element from a sequence like a list or string. It returns one item from the given sequence.
4. random.random() does not take range values like randint. It always works between 0 and 1.
5. These functions are part of the random module. They are used in games, simulations, and data sampling

13. Explain the concept of Default Arguments with a code example.

1. Default arguments are parameters that have a predefined value in a function. If no value is given during the function call, the default value is used.
2. They help make functions flexible and easier to use. We do not need to pass all arguments every time.
3. Syntax example:

```
def greet(name="Guest"):
  print("Hello", name)
```

4. If we call greet("Meet"), it prints Hello Meet. If we call greet(), it prints Hello Guest.
5. Default arguments must be written after normal parameters. They reduce errors and make the function simple to handle.

---

14. Mention any two constants available in the string module.

1. string.ascii_letters contains all lowercase and uppercase letters. It includes characters from a to z and A to Z.
2. string.digits contains all numeric characters from 0 to 9. It is useful when working with numbers in string format.
3. These constants are available in the string module. We must write import string before using them.
4. They help in tasks like password generation and validation. They save time by providing ready-made character sets.
5. The string module mainly provides constants and helper tools. It does not directly change string values like built-in methods.

---

15. Write a code to display the current system date and time.

1. The datetime module is used to get the current system date and time. It provides the datetime class for this purpose.
2. We must first import the module before using it. This can be done using from datetime import datetime.
3. Example code:

```
from datetime import datetime
now = datetime.now()
print("Current Date and Time:", now)
```

4. The function datetime.now() returns the current local date and time. It stores the value in the variable now.
5. When we print the variable, it shows the full timestamp. This is useful in logging and real-time applications.

(5 marks)

1. Explain the different types of function arguments in Python with examples.

1. Positional arguments are passed in the correct order to the function. For example, add(2,3) sends 2 and 3 in order.
2. Keyword arguments are passed using key = value format. For example, add(a=2, b=3) clearly assigns values to parameters.
3. Default arguments have predefined values in the function definition. For example, def greet(name="Guest") uses Guest if no value is given.
4. Variable length arguments use *args to accept multiple positional values. For example, def sum(*nums) can take many numbers.
5. Keyword variable length arguments use **kwargs to accept multiple keyword values. For example, def info(**data) stores values in dictionary form.
6. These argument types make functions flexible and powerful. They allow different ways of sending data to functions.

2. Write a Python program to calculate the factorial of a number using a user-defined function.

1. Factorial of a number means multiplying all numbers from 1 to that number. For example, factorial of 5 is 5 × 4 × 3 × 2 × 1.
2. We create a user-defined function using the def keyword. The function will take a number as input.
3. Example program:

```
def factorial(n):
  fact = 1
  for i in range(1, n+1):
    fact = fact * i
  return fact
```

4. Then we call the function and print the result. For example, print(factorial(5)).
5. The loop multiplies each number step by step. The result is returned using the return statement.
6. This program shows how user-defined functions help reuse logic. It makes the code clear and organized.

3. Describe how to import a module in Python using different methods with examples.

1. We can import the entire module using import module_name. For example, import math allows access to all math functions.
2. After importing the full module, we must use dot notation. For example, math.sqrt(16) returns 4.
3. We can import specific functions using from module_name import function_name. For example, from math import sqrt.
4. In this case, we can directly write sqrt(16) without using math. This makes the code shorter.
5. We can also use alias names using import module_name as alias. For example, import math as m allows writing m.sqrt(16).
6. Another method is from module_name import * which imports all names. But this method is not recommended because it may create confusion.

---

4. Write a program that uses the math module to calculate square root, power, and logarithm of a number.

1. The math module provides many mathematical functions. We must first import it using import math.
2. The function math.sqrt() is used to find square root. The function math.pow() is used to find power.
3. The function math.log() is used to find the natural logarithm of a number. It returns the log value.
4. Example program:

```
import math
num = 16
print("Square Root:", math.sqrt(num))
print("Power:", math.pow(num, 2))
print("Logarithm:", math.log(num))
```

5. The program calculates square root, power, and log of the number 16. Each result is printed separately.
6. This program shows how useful the math module is. It makes complex calculations simple and fast.

5. Explain the working of the random module with suitable examples.

1. The random module is used to generate random numbers and random selections in Python. It is commonly used in games, simulations, and testing.
2. We must first import the module using import random. After importing, we can use different random functions.
3. The function random.random() returns a random float value between 0 and 1. For example, random.random() may return 0.56.
4. The function random.randint(a, b) returns a random integer between a and b. For example, random.randint(1,10) may return 7.
5. The function random.choice(seq) selects a random element from a sequence. For example, random.choice([1,2,3]) may return 2.
6. Each time we run the program, the output may change. This is because the numbers are generated randomly.

---

6. Write a Python program to display the current date and time using the datetime module.

1. The datetime module is used to work with date and time in Python. It provides classes like date and datetime.
2. To use it, we must import it first. This can be done using from datetime import datetime.
3. The function datetime.now() gives the current system date and time. It returns a complete timestamp.
4. Example program:

```
from datetime import datetime
now = datetime.now()
print("Current Date and Time:", now)
```

5. The value returned is stored in the variable now. When printed, it shows year, month, day, hour, minute, and seconds.
6. This program is useful for logging and real-time applications. It helps in tracking time-based events.

---

7. Explain important string functions such as upper(), lower(), split(), and replace() with examples.

1. The upper() function converts all letters of a string to uppercase. For example, "hello".upper() gives HELLO.
2. The lower() function converts all letters of a string to lowercase. For example, "HELLO".lower() gives hello.
3. The split() function divides a string into a list based on a separator. For example, "a,b,c".split(",") gives ['a','b','c'].
4. The replace() function replaces one part of a string with another. For example, "cat".replace("c","b") gives bat.
5. These functions help in modifying and processing string data. They are built-in string methods in Python.
6. String functions make text handling simple and efficient. They are widely used in real-world applications.

---

8. Write a function to check whether a number is prime or not.

1. A prime number is a number greater than 1 that has only two factors, 1 and itself. It cannot be divided by any other number.
2. We create a user-defined function using the def keyword. The function will take a number as input.
3. Example program:

```
def is_prime(n):
  if n <= 1:
    return False
  for i in range(2, n):
    if n % i == 0:
      return False
  return True
```

4. The loop checks if the number is divisible by any value between 2 and n-1. If it is divisible, the function returns False.
5. If no divisor is found, the function returns True. This means the number is prime.
6. We can test it using print(is_prime(7)). The output will be True because 7 is a prime number.

9. Explain the steps to create a custom module and import it into another Python file.

1. A custom module is a Python file that contains functions or variables. We create it by saving code in a file with a .py extension.
2. For example, create a file named mymodule.py. Inside it, define a function like def greet(): print("Hello").
3. Save this file in the same folder as your main Python file. This allows Python to find the module easily.
4. In another Python file, write import mymodule to import the module. Then call the function using mymodule.greet().
5. You can also import specific functions using from mymodule import greet. Then you can directly write greet().
6. Custom modules help organize code into separate files. They make programs easier to manage and reuse.

---

10. Write a program to simulate a dice roll (generate a random number between 1 and 6).

1. To simulate a dice roll, we use the random module. It helps generate random numbers.
2. First, we import the module using import random. This allows us to use its functions.
3. We use the function random.randint(1, 6) to generate a number. It gives a random integer between 1 and 6.
4. Example program:

```
import random
dice = random.randint(1, 6)
print("Dice roll:", dice)
```

5. Each time the program runs, it shows a different number. This makes it similar to rolling a real dice.
6. This program is useful in simple games and simulations. It shows how randomness works in Python.

---

11. What are anonymous functions? Write a lambda function to find the square of a number.

1. Anonymous functions are functions without a name. They are created using the lambda keyword.
2. Lambda functions are usually small and written in one line. They are used for short operations.
3. Syntax of lambda function is lambda arguments: expression. It returns the result automatically.
4. Example to find square:

```
square = lambda x: x * x
print(square(5))
```

5. In this example, x is the input parameter. The expression x * x calculates the square.
6. Lambda functions are useful for quick calculations. They are often used with functions like map() and filter().

---

12. Write a program to calculate the number of days remaining until a specific future date (e.g., New Year's Eve).

1. We use the datetime module to work with dates. It helps calculate the difference between two dates.
2. First, we import the required classes using from datetime import datetime. This allows us to use date and time features.
3. We create an object for today's date using datetime.now(). Then we create another object for the future date.
4. Example program:

```
from datetime import datetime

today = datetime.now()
future_date = datetime(2026, 12, 31)

remaining = future_date - today
print("Days remaining:", remaining.days)
```

5. The subtraction of two dates gives a timedelta object. The .days attribute shows the number of days left.
6. This program is useful for countdowns and planning events. It helps calculate time differences easily.

(10 – marks)

1. Explain user-defined functions in Python. Discuss function definition, function call, return statement, and types of arguments with examples.

1. A user-defined function is a function created by the programmer to perform a specific task. It helps in dividing a large program into smaller and manageable parts.
2. A function is defined using the def keyword followed by the function name and parentheses. The syntax is def function_name(parameters): followed by indented statements.
3. Example of function definition is:

```
def greet():
  print("Hello")
```

This creates a function named greet.

4. A function call means executing the function. We call it by writing its name followed by parentheses like greet().
5. The return statement is used to send a result back from the function. For example, return a + b sends the calculated value.
6. Functions can have different types of arguments. Positional arguments are passed in order like add(2,3).
7. Default arguments have predefined values like def greet(name="Guest"). If no value is given, it uses the default value.
8. Variable length arguments like *args and **kwargs allow passing multiple values. These argument types make functions flexible and powerful.

2. Write a Python program using functions to perform addition, subtraction, multiplication, and division based on user choice.

1. We can create separate functions for addition, subtraction, multiplication, and division. This makes the program organized and easy to understand.

program:

```
def add(a, b):
   return a + b

def subtract(a, b):
   return a - b

def multiply(a, b):
   return a * b

def divide(a, b):
   return a / b

print("1. Addition")
print("2. Subtraction")
print("3. Multiplication")
print("4. Division")
choice = int(input("Enter your choice (1-4): "))
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

if choice == 1:
   print("Result:", add(num1, num2))
elif choice == 2:
   print("Result:", subtract(num1, num2))
elif choice == 3:
   print("Result:", multiply(num1, num2))
elif choice == 4:
   print("Result:", divide(num1, num2))
else:
   print("Invalid choice")
```

3. Explain the datetime module in detail. Write a program to display current date, time, and formatted output.

1. The datetime module is used in Python to work with date and time. It helps in handling real-world date and time operations easily.
2. It provides important classes like date, time, datetime, and timedelta. These classes help create, format, and calculate date and time values.
3. The datetime.now() function is used to get the current system date and time. The date.today() function returns the current date only.
4. We must first import the module before using it. This can be done using from datetime import datetime.
5. The module also allows formatting using the strftime() function. This converts date and time into a readable string format.
6. Example program:

```
from datetime import datetime

now = datetime.now()

print("Current Date and Time:", now)
print("Current Date:", now.date())
print("Current Time:", now.time())
print("Formatted Date:", now.strftime("%d-%m-%Y"))
print("Formatted Time:", now.strftime("%H:%M:%S"))
```

7. In this program, now stores the current date and time. The strftime() method formats the output in a custom style.
8. The datetime module is very useful in logging, scheduling, and time calculations. It makes date and time handling simple and efficient.

4. Write a Python program that uses the random module to simulate rolling a dice 10 times and display the results.

1. The random module is used to generate random numbers in Python. It is commonly used in games and simulations.
2. To simulate a dice roll, we use random.randint(1, 6). It generates a random number between 1 and 6.
3. First, we must import the module using import random. Then we can use its functions.
4. Example program:

```
import random

print("Rolling dice 10 times:")
for i in range(10):
   dice = random.randint(1, 6)
   print("Roll", i+1, ":", dice)
```

5. The loop runs 10 times to simulate 10 dice rolls. Each time it prints a new random number.
6. Every time the program runs, the results may be different. This is because the numbers are generated randomly.
7. This program is useful for understanding randomness. It shows how loops and modules work together.
8. The random module makes it easy to simulate real-life random events. It is simple and powerful to use.

5. Explain the math module and demonstrate at least five mathematical functions with a Python program.

1. The math module is a built-in module in Python that provides mathematical functions. It helps perform calculations like square root, power, logarithm, and trigonometric operations.
2. To use the math module, we must first import it using import math. After importing, we can access functions using dot notation like math.sqrt().
3. The function math.sqrt(x) returns the square root of a number. The function math.pow(x, y) returns x raised to the power y.
4. The function math.log(x) returns the natural logarithm of a number. The function math.ceil(x) rounds a number up to the nearest integer.
5. The function math.floor(x) rounds a number down to the nearest integer. These functions make mathematical calculations simple and fast.
6. Example program:

```
import math

num = 16

print("Square Root:", math.sqrt(num))
print("Power (num^2):", math.pow(num, 2))
print("Logarithm:", math.log(num))
print("Ceiling of 4.3:", math.ceil(4.3))
print("Floor of 4.8:", math.floor(4.8))
```

7. In this program, different math functions are applied to numbers. Each function performs a specific mathematical operation.
8. The math module is useful in scientific and engineering applications. It reduces the need to write complex formulas manually.

6. Write a Python program that uses string functions to count vowels, convert case, and reverse a string.

1. String functions help in modifying and analyzing text data. They are built-in methods available in Python.
2. We can use lower() and upper() functions to convert the case of a string. These functions change letters to lowercase or uppercase.
3. To count vowels, we can check each character in the string. We compare it with vowels like a, e, i, o, u.
4. To reverse a string, we can use slicing method [::-1]. This returns the string in reverse order.
5. Example program:

```
text = input("Enter a string: ")

vowels = "aeiouAEIOU"
count = 0

for ch in text:
   if ch in vowels:
      count += 1

print("Number of vowels:", count)
print("Uppercase:", text.upper())
print("Lowercase:", text.lower())
print("Reversed string:", text[::-1])
```

6. The loop checks each character to count vowels. The case conversion and reversing are done using built-in string methods.
7. This program shows how strings can be processed easily. It uses simple and useful string operations.
8. String functions are widely used in text processing and data handling. They make working with text very simple.

7. Compare built-in functions and user-defined functions. Write programs demonstrating both.

1. Built-in functions are already available in Python. User-defined functions are created by the programmer.
2. Built-in functions do not need to be defined before use. User-defined functions must be defined using the def keyword.
3. Examples of built-in functions are len(), type(), and sum(). These functions perform common tasks easily.
4. Example program using built-in functions:

```
numbers = [1, 2, 3, 4, 5]
print("Length:", len(numbers))
print("Sum:", sum(numbers))
print("Type:", type(numbers))
```

5. User-defined functions are used when we want custom logic. They make programs modular and organized.
6. Example program using a user-defined function:

```
def greet(name):
   return "Hello " + name

print(greet("Meet"))
```

7. Built-in functions save time and reduce code length. User-defined functions give flexibility to solve specific problems.
8. Both types of functions are important in programming. They help make the code simple and reusable.

8. Develop a Python program that uses multiple modules (math, random, datetime) to create a simple number-guessing game with time tracking.

1. In this program, we use the random module to generate a secret number. We use the datetime module to track the time taken by the player.

program:

```
import random
import math
from datetime import datetime

secret = random.randint(1, 50)
start_time = datetime.now()

print("Guess the number between 1 and 50")

while True:
  guess = int(input("Enter your guess: "))

  if guess == secret:
    print("Correct guess!")
    break
  else:
    diff = math.fabs(secret - guess)
    print("Wrong guess. Difference is:", diff)

end_time = datetime.now()
time_taken = end_time - start_time

print("Time taken (seconds):", time_taken.total_seconds())
```

2. The program keeps running until the correct number is guessed. It shows the difference to help the player guess better.
3. The datetime module calculates how long the player took. It subtracts start time from end time.
4. The random module ensures the number changes every time the game runs. This makes the game interesting.
5. This program shows how multiple modules can work together. It combines randomness, calculations, and time tracking in one application.

9. (a) Describe the different types of arguments in Python: Positional, Keyword, Arbitrary (*args), and Keyword-Arbitrary (kwargs).**

(b) Write a function that accepts any number of numeric arguments and returns their sum.

1. Positional arguments are passed in the correct order to a function. For example, add(2,3) sends values based on position.
2. Keyword arguments are passed using key = value format. For example, add(a=2, b=3) clearly assigns values to parameters.
3. Arbitrary arguments use *args to accept any number of positional arguments. Inside the function, they are stored as a tuple.
4. Keyword-arbitrary arguments use **kwargs to accept any number of keyword arguments. Inside the function, they are stored as a dictionary.
5. These argument types make functions flexible and powerful. They allow us to handle different types of input easily.
6. Example function to return sum of any number of numeric arguments:

```
def total_sum(*numbers):
  total = 0
  for num in numbers:
    total += num
  return total

print(total_sum(2, 4, 6, 8))
```

7. The *numbers collects all values passed into the function. The loop adds each number to the total.
8. This function works for any number of values. It demonstrates the use of arbitrary arguments clearly.

10. (a) Explain the concept of "seeding" in random number generation using random.seed().

(b) Write a Python program to generate a random password of length 8.

1. Seeding means setting a starting point for random number generation. It is done using random.seed().
2. When we use the same seed value, the random numbers generated will be the same each time. This is useful for testing and debugging.
3. Example: random.seed(10) will always generate the same sequence of random numbers. Without seed, numbers change every time.
4. Seeding helps make random results predictable when needed. It controls how the random module generates numbers.
5. To generate a random password, we can use letters and digits. We combine characters and select randomly.
6. Example program:

```
import random
import string

characters = string.ascii_letters + string.digits
password = ""

for i in range(8):
   password += random.choice(characters)

print("Random Password:", password)
```

7. The program selects 8 random characters from letters and digits. The loop runs 8 times to build the password.
8. Each time the program runs, a different password is generated. This shows practical use of the random module.

11. (a) Explain how Python locates modules (the sys.path mechanism).

(b) Discuss the difference between a Module and a Package.

1. When we import a module, Python searches for it in a list of directories. This list is stored in sys.path.
2. The sys.path list includes the current directory and standard library directories. Python checks these locations one by one.
3. If the module file is found in any directory in sys.path, it is loaded. If not found, Python gives a ModuleNotFoundError.
4. We can view the search path using import sys and print(sys.path). This shows all directories where Python searches.
5. A module is a single Python file with a .py extension. It contains functions, classes, or variables.
6. A package is a collection of related modules stored in a folder. It usually contains an __init__.py file.
7. Modules help organize code into separate files. Packages help organize multiple modules into a structured format.
8. In simple words, a module is one file, while a package is a folder of modules. Both help in better code management and reuse.

12. (a) Define recursion.

(b) Write a recursive function to find sum of N natural numbers.

1. Recursion is a technique where a function calls itself. It is used to solve problems that can be divided into smaller similar problems.
2. In recursion, there must be a base case. The base case stops the function from calling itself again.
3. Without a base case, recursion will run forever. This may cause a stack overflow error.
4. To find the sum of N natural numbers, we can use recursion. The function will add the number to the sum of previous numbers.
5. Example recursive function:

```
def sum_n(n):
  if n == 1:
    return 1
  else:
    return n + sum_n(n-1)

print(sum_n(5))
```

6. In this function, when n becomes 1, it returns 1. This is the base case.
7. For other values, the function calls itself with n-1. It keeps adding numbers until it reaches the base case.
8. This program calculates the sum step by step using recursion. It clearly shows how recursive functions work in Python.

13. (a) Discuss the string module's capabilities.

(b) Write a function to generate a secure random password of 10 characters containing uppercase, lowercase, digits, and symbols.

1. The string module in Python provides useful string constants and helper tools. It includes predefined character sets like letters, digits, and punctuation.
2. Some important constants are string.ascii_letters, string.digits, and string.punctuation. These constants save time because we do not need to type characters manually.
3. The module is helpful in tasks like password generation and validation. It is also useful in text processing applications.
4. To use the string module, we must import it using import string. After importing, we can access its constants using dot notation.
5. The string module mainly provides constants, not string modification methods. String modification is done using built-in methods like upper() and lower().
6. To generate a secure password, we combine uppercase, lowercase, digits, and symbols. We also use the random module to select characters randomly.
7. Example function to generate a secure password of length 10:

```
import random
import string

def generate_password():
    characters = string.ascii_letters + string.digits + string.punctuation
    password = ""
    for i in range(10):
        password += random.choice(characters)
    return password

print("Secure Password:", generate_password())
```

8. This function selects 10 random characters from the combined set. Each time the program runs, it generates a different secure password.

14. Create a program that uses random, math, and datetime modules.

1. This program will use the random module to select a random mathematical operation. It will choose between Add, Subtract, and Multiply.
2. It will also generate two random numbers for the calculation. These numbers will be used as inputs for the selected operation.
3. The math module will be used to perform extra calculations like square root and power. We will use math.sqrt() and math.pow() functions.
4. The datetime module will display the execution timestamp. This shows when the program was executed.

```
import random
import math
from datetime import datetime

now = datetime.now()
print("Execution Time:", now.strftime("%d-%m-%Y %H:%M:%S"))

num1 = random.randint(1, 20)
num2 = random.randint(1, 20)

operation = random.choice(["Add", "Sub", "Mul"])
print("Numbers:", num1, num2)
print("Selected Operation:", operation)

if operation == "Add":
  result = num1 + num2
elif operation == "Sub":
  result = num1 - num2
else:
  result = num1 * num2

print("Result:", result)
print("Square Root of num1:", math.sqrt(num1))
print("num1 raised to power 2:", math.pow(num1, 2))
```

5. The program first shows the current date and time using strftime(). Then it randomly selects numbers and an operation.
6. Finally, it performs the selected operation and additional math calculations. This example shows how multiple modules can work together in one program.

15. (a) Explain the strftime() and strptime() methods with examples for formatting and parsing dates.

(b) Write a program to calculate a person's exact age in years, months, and days based on their date of birth.

1. The strftime() method is used to convert a date or datetime object into a formatted string. It helps display date and time in a readable format like day-month-year.
2. For example, now.strftime("%d-%m-%Y") formats the date as 03-03-2026. It uses format codes like %d for day, %m for month, and %Y for year.
3. The strptime() method is used to convert a string into a datetime object. It parses the string according to the given format.
4. For example, datetime.strptime("03-03-2026", "%d-%m-%Y") converts the string into a date object. This is useful when taking date input from the user.
5. Both methods are part of the datetime module. They are commonly used for formatting and parsing date values.
6. To calculate exact age, we first take the date of birth from the user. Then we convert it into a date object using strptime().
7. We compare the birth date with today's date to calculate the difference. Then we adjust years, months, and days properly.
8. Example program:

```
from datetime import datetime

dob_input = input("Enter Date of Birth (DD-MM-YYYY): ")
dob = datetime.strptime(dob_input, "%d-%m-%Y")
today = datetime.now()

years = today.year - dob.year
months = today.month - dob.month
days = today.day - dob.day

if days < 0:
  months -= 1
  days += 30

if months < 0:
  years -= 1
  months += 12

print("Exact Age:", years, "Years,", months, "Months,", days, "Days")
```